STUDENT MANAGEMENT SYSTEM

Python Console Application

Internship	InternNova — Week 6 Assignment
Project type	Console-based student record management
Language	Python 3
Data storage	JSON file (students.json)
Repository	github.com/160723735020-pixel/InternNova-Student-Management

Project Report

Prepared as documentation for the InternNova Week 6 Python project submission.

1. Executive Summary

The Student Management System is a lightweight, menu-driven Python application for maintaining student records from a console. It supports adding, viewing, searching, updating, and deleting students, as well as calculating a grade from marks. Student data is persisted locally in a JSON file so that records can be loaded again when the application starts. The project uses separate Python modules to keep student data representation, record-management operations, and the menu interface organized.

2. Problem Statement

Managing student details manually can lead to inconsistent records and makes common operations repetitive. This project demonstrates a small software solution that provides a structured way to maintain student identifiers, names, ages, courses, and marks. The scope is intentionally limited to a console application and local file storage, making it easy to run and understand without external services or database setup.

3. Objectives

• Provide a clear, menu-driven interface for student record operations.

• Represent each student using a Python class and encapsulated attributes.

• Implement CRUD operations: create, read, update, and delete student records.

• Calculate a letter grade from the student's marks using defined thresholds.

• Persist records using JSON and handle invalid input or file-related errors.

• Organize code into modules and publish the project through Git and GitHub.

4. Technologies and Project Structure

The application uses Python's standard library and does not require third-party packages for its core functionality. JSON is used for data persistence and pathlib is used to locate the data file relative to the project code.

File / folder	Purpose
main.py	Console menu, user input, and application flow
student.py	Student class, properties, serialization helpers, and display behavior
student_manager.py	Loading/saving JSON and managing student records
data/students.json	Local JSON data store; cleared to an empty list for submission
screenshots/	Screenshots documenting the application's features and test runs
README.md	Project overview, setup instructions, and usage documentation
requirements.txt	Notes that no external dependencies are required
.gitignore	Excludes Python cache and common local development files

5. System Design

The application is divided into three main responsibilities:

Student model: stores student attributes and provides grade calculation, dictionary conversion, reconstruction from stored data, and display behavior.

Student manager: maintains students indexed by student ID, checks for duplicate IDs, performs record operations, and reads/writes the JSON file.

Console interface: displays the menu, collects user input, validates values, and calls the manager's operations.

Typical flow: user selects a menu action → the interface validates input → the manager performs the requested operation → changes are saved to JSON when appropriate → the result is displayed in the console.

6. Features and Functional Requirements

Feature	Description
Add student	Collects student ID, name, age, course, and marks, then saves a new record.
View all students	Displays stored student records.
Search student	Finds a student using the student ID.
Update student	Allows an existing record's details to be changed.
Delete student	Removes a selected student record.
Calculate grade	Maps marks to a letter grade using the project's grade thresholds.
Persistence	Loads saved records at startup and stores changes in data/students.json.
Validation	Checks menu choices, required text fields, age range (1–120), and marks range (0–100).

7. Grade Calculation Rules

The application assigns a grade according to the following thresholds. These are the rules implemented in the project.

Marks	Grade
90–100	A+
80–89.99	A
70–79.99	B
60–69.99	C
50–59.99	D
Below 50	F

8. Python Concepts Demonstrated

• Variables, conditional logic, loops, and functions.

• Classes and objects through the Student and StudentManager classes.

• Encapsulation using attributes and properties.

• Collections for keeping records indexed by student ID.

• Modules and imports to separate responsibilities.

• File handling and JSON serialization for persistence.

• Exception handling and input validation for more robust interaction.

9. Testing and Results

During manual testing, the application was run from PowerShell. The menu and exit path were verified, saved student data was loaded from JSON, and the core record-management workflow and grade calculation were exercised with a sample record. Invalid menu input was also checked. The screenshots directory contains evidence captured during these runs.

The final data file was reset to an empty JSON list ([]) so the submitted repository starts without demonstration records. Screenshots may show sample data from test runs; these are included to demonstrate behavior rather than represent the final stored database.

Suggested final test checklist

• Add a student with valid values and confirm that the record is saved.

• View all records and search using a known student ID.

• Update a record and confirm the new values persist.

• Delete a record and verify that it no longer appears.

• Check grade boundaries, including 90, 80, 70, 60, 50, and a value below 50.

• Try invalid menu input, empty text, out-of-range age, and marks outside 0–100.

10. Git and GitHub

The project was initialized as a Git repository, the branch was named main, and the initial commit was created with the message “Initial commit: Student Management System”. A remote named origin was configured for the public GitHub repository. The first push completed successfully and established main as the upstream branch.

Repository: https://github.com/160723735020-pixel/InternNova-Student-Management

11. Challenges and Future Improvements

Challenges addressed: keeping student data after the application exits, validating console input, handling JSON data files, and configuring GitHub authentication on Windows.

Possible future improvements: add automated unit tests, provide CSV export, introduce a graphical interface, use a database for larger datasets, and add stronger validation for duplicate or malformed data. These are potential enhancements and are not claimed as current features.

12. Conclusion

The Student Management System demonstrates how Python classes, modular design, validation, and JSON file handling can be combined to build a useful console application. It implements the core student-record workflow and is maintained in a public GitHub repository with a README and screenshots. The project remains deliberately small and suitable for demonstrating fundamental Python programming and version-control practices.

Submission note: Before submitting, verify that the repository URL is accessible and that all screenshots and project files are present.